Share

Ok, we're starting over from the top. Please run Phase 1 from ProjectKickoff and either write markdown files directly to the project if you can, or create them here that I can copy.

Thought for 5s

I can’t write directly into your local project, but I can generate the Phase 1 files exactly as markdown so you can drop them into the repo.

Based on the ProjectKickoff structure we discussed, I'd create:

```
Docs/
└── Product/
    ├── ProductVision.md
    ├── ProblemStatement.md
    └── MVPDefinition.md
```

ProductVision.md

Edit

Product Vision

Product Name

NextSeason

Vision

NextSeason helps television viewers keep track of the shows they care about and notifies them when a new season becomes available.

Many viewers enjoy serialized television but struggle to remember when shows return after long breaks between seasons. Existing solutions are often focused on episode tracking, ratings, reviews, social features, or streaming discovery. These features add complexity and do not directly solve the core problem.

NextSeason focuses on a single job:

> Tell me when the next season of my favorite show is available.

Target Audience

Primary audience:

- People who follow multiple television series
- Viewers frustrated by long gaps between seasons
- Users who want a simple watchlist rather than a full media management platform

Secondary audience:

- Casual viewers who only track a handful of shows
- Users who have abandoned larger tracking applications because they are too complicated

---

Core Value Proposition

Instead of requiring users to regularly search for updates, NextSeason continuously monitors tracked shows and notifies users when new seasons become available.

The application should provide value immediately after setup:

1. Search for a show.
2. Add it to a watchlist.
3. Receive a notification when a new season becomes available.

Product Principles

Simplicity First

Every feature must support the primary job of tracking television seasons.

Low Maintenance

Users should not need to manually check for updates.

Minimal User Effort

Adding a show should require only a few taps.

Reliable Information

Season information should come from a reputable television data source.

Focused Scope

Features that do not directly improve season tracking should be considered carefully before inclusion.

Success Criteria

A user can:

- Find a show quickly
- Add it to a watchlist
- Understand when the next season is expected
- Receive a notification when the season becomes available

If users successfully receive and act on notifications, the product has achieved its primary purpose.

ProblemStatement.md

Edit

Problem Statement

The Problem

Television shows often have long and unpredictable gaps between seasons.

A viewer may finish a season and then wait months or years for the next one. During that time it is easy to forget:

- Whether the show was renewed
- When the next season is expected
- Whether the new season has already been released

As viewers follow more shows, this problem becomes increasingly difficult to manage manually.

Current Alternatives

Users typically rely on one or more of the following approaches:

- Memory
- Calendar reminders
- Internet searches
- Entertainment news sites
- Social media
- Large television tracking applications

These approaches have several drawbacks:

- Require active effort from the user
- Produce inconsistent results
- Create information overload

- Mix season tracking with many unrelated features

User Needs

Users need a simple way to:

- Track television shows they care about
- Know whether a show has been renewed
- Know when a new season is expected
- Be notified when a season becomes available

The solution should require little ongoing maintenance.

Why This Matters

Television content is increasingly fragmented across streaming platforms.

Viewers can no longer rely on traditional seasonal schedules and often lose track of shows they intended to continue watching.

A focused solution that monitors television series and provides timely notifications removes this burden from the user.

MVPDefinition.md

Edit

MVP Definition

Goal

Validate that users find value in receiving notifications when new seasons of tracked television shows become available.

The MVP should solve a single problem well before expanding into adjacent areas.

In Scope

Show Search

Users can search for television shows.

Requirements:

- Search by title
- Display matching results
- Display basic show information

Watchlist

Users can add shows to a watchlist.

Requirements:

- Add show
- Remove show
- View tracked shows

Season Status

Users can see season information.

Requirements:

- Latest known season
- Upcoming season information when available
- Status indicators such as:
  - Returning
  - Running
  - Ended

Notifications

Users receive notifications when a new season becomes available.

Requirements:

- Local notification delivery
- Notification content identifies the show
- User can open the app from the notification

Background Updates

The app periodically refreshes show information.

Requirements:

- Check tracked shows
- Detect season changes
- Trigger notifications when appropriate

Out of Scope

The MVP will NOT include:

- Episode tracking
- Ratings
- Reviews
- Recommendations
- Social features
- Friends or sharing
- Streaming service availability
- User accounts
- Cloud synchronization
- Cross-platform support
- Android application
- Web application
- Premium subscriptions
- Advertising

Success Metrics

The MVP is successful if users can:

1. Add shows to a watchlist.
2. Leave the application.
3. Receive notifications when a new season becomes available.
4. Continue using the application without ongoing manual effort.

MVP Exit Criteria

The MVP is complete when:

- Search works reliably
- Watchlist management works reliably
- Season data updates correctly
- Notifications are delivered correctly
- The app can be demonstrated as a portfolio-quality project
- The project demonstrates effective AI-assisted development practices

These are the three documents I would consider the complete Phase 1 foundation. After they're committed, Phase 2 would be turning the MVP definition into concrete user stories and acceptance criteria before any architecture work begins.

I don't see anything here about logging in. If we're letting users add shows to a watchlist then we need a way for them to log in.

You're right. I scoped it as **local-only MVP**, but I didn't say that explicitly, which is a gap.

For MVP we should choose one:

Option A — No login for MVP
Watchlist is stored locally on-device. Simpler, faster, better for Phase 1.

Option B — Login included in MVP
Needed if we want cloud sync, account recovery, multiple devices, or server-side notification tracking.

My recommendation: **No login in MVP**, but document it clearly:

Markdown

```
## Accounts and Login

The MVP will not include user accounts or login.

User watchlists will be stored locally on the device.

This keeps the first version simple and avoids adding authentication, backend user management, acco

Login may be added in a later phase if the product expands to support cloud sync, multi-device use,
```

And update **Out of Scope** to say:

Markdown

```
- User accounts
- Login
- Cloud synchronization
- Multi-device sync
```

So yes: either we add login to scope, or we explicitly state that watchlists are local-only. I think local-only is the better MVP.

go ahead with phase 2

Perfect. Phase 2 should transform the MVP into **concrete user stories and acceptance criteria**.

I'd create:

```
Docs/
└── Product/
    ├── UserStories.md
    ├── FunctionalRequirements.md
    └── MVPBacklog.md
```

UserStories.md

Markdown

```
# User Stories

## Epic: Show Discovery

### US-001 Search for a Show

As a user,
I want to search for a TV show by name,
so that I can add it to my watchlist.

#### Acceptance Criteria

- User can enter text into a search field.
- Search results appear within a reasonable time.
- Results include:
  - Show title
  - Poster image (if available)
  - Show status
- User can select a show from results.

---

### US-002 View Show Details

As a user,
I want to view details about a show,
so that I can determine whether it is the correct show.

#### Acceptance Criteria

- User can open a show detail screen.
- Screen displays:
  - Title
  - Description
  - Poster
  - Status
  - Most recent season information
- User can add the show to their watchlist.

---

## Epic: Watchlist Management

### US-003 Add Show to Watchlist

As a user,
I want to add a show to my watchlist,
```

```
so that I can track future seasons.

#### Acceptance Criteria

- User can add a show from search results.
- User can add a show from show details.
- Added show immediately appears in watchlist.
- Duplicate additions are prevented.

---

### US-004 Remove Show from Watchlist

As a user,
I want to remove a show from my watchlist,
so that I stop receiving notifications.

#### Acceptance Criteria

- User can remove a tracked show.
- Removed shows no longer receive updates.
- Removed shows no longer generate notifications.

---

### US-005 View Watchlist

As a user,
I want to see all tracked shows,
so that I know what I am monitoring.

#### Acceptance Criteria

- Watchlist displays all tracked shows.
- Watchlist displays current status.
- User can tap a show for details.

---

## Epic: Season Tracking

### US-006 View Season Status

As a user,
I want to see the current status of a show,
so that I know whether a new season is expected.

#### Acceptance Criteria

- Show displays status:
  - Running
  - Returning
  - Ended
  - Unknown
- Last known season is visible.
- Next season information is shown when available.

---

## Epic: Notifications
```

```
### US-007 Receive Notification

As a user,
I want to receive a notification when a new season becomes available,
so that I do not miss it.

#### Acceptance Criteria

- App requests notification permission.
- Notification identifies the show.
- Tapping notification opens the app.

---

### US-008 Manage Notification Permission

As a user,
I want to control notifications,
so that the app respects my preferences.

#### Acceptance Criteria

- App handles denied permission gracefully.
- App explains why notifications are useful.
- User can continue using the app without notifications.
```

FunctionalRequirements.md

Markdown

```
# Functional Requirements

## Search

FR-001
The system shall allow users to search for television shows by title.

FR-002
The system shall display search results returned by the selected TV data provider.

FR-003
The system shall display show artwork when available.

---

## Watchlist

FR-004
The system shall allow users to add shows to a watchlist.

FR-005
The system shall prevent duplicate watchlist entries.

FR-006
The system shall allow users to remove shows from a watchlist.
```

```
FR-007
The system shall persist watchlist data between launches.

---

## Show Details

FR-008
The system shall display basic metadata for a show.

FR-009
The system shall display current show status.

FR-010
The system shall display season information when available.

---

## Notifications

FR-011
The system shall request notification permission.

FR-012
The system shall generate notifications when a tracked show receives a new season.

FR-013
The system shall avoid duplicate notifications for the same season.

---

## Data Refresh

FR-014
The system shall periodically refresh tracked show data.

FR-015
The system shall compare refreshed data against previously known data.

FR-016
The system shall detect newly available seasons.

---

## Storage

FR-017
The system shall store watchlist information locally.

FR-018
The system shall persist notification history required to prevent duplicates.
```

MVPBacklog.md

Markdown

```
# MVP Backlog
```

```
# MVP Backlog

## P0 — Required for MVP

### Search

- Search screen
- Search API integration
- Search results list

### Show Details

- Show detail screen
- Show metadata display
- Add to watchlist

### Watchlist

- Watchlist screen
- Add show
- Remove show
- Local persistence

### Notifications

- Notification permission flow
- Local notifications
- Notification handling

### Data Refresh

- Background refresh mechanism
- Season change detection
- Notification generation

---

## P1 — Nice to Have

### Watchlist Improvements

- Sort watchlist
- Filter watchlist
- Search within watchlist

### UI Polish

- Better loading states
- Empty state illustrations
- Enhanced animations

---

## P2 — Future Versions

### Accounts

- User registration
- Login
- Password reset
```

```
### Cloud Sync

- Multi-device synchronization
- Backup and restore

### Streaming Information

- Streaming provider availability
- Direct deep links

### Social Features

- Sharing
- Recommendations

### Monetization

- Subscription tier
- Premium features
```

One thing I'd change from my earlier Phase 1 recommendation: now that you've called out login, I'd add a **Decision Log** document and record:

Markdown

```
# Product Decisions

## PD-001 User Accounts

Decision:
No user accounts in MVP.

Rationale:
The MVP validates demand for season notifications.
Local storage dramatically reduces complexity and allows focus on the core value proposition.

Future Revisit:
When cloud sync or multi-device support becomes necessary.
```

That gives future-you (and future AI agents) a record of *why* login isn't present, rather than looking like we